Arithmetic Logic Unit

Unità Artmetico Logica (ALU)

- Deve eseguire le operazioni previste dall'ISA

  `add, addi, addiu, addu`

  `sub, subu`

  `and, andi, nor, or,`

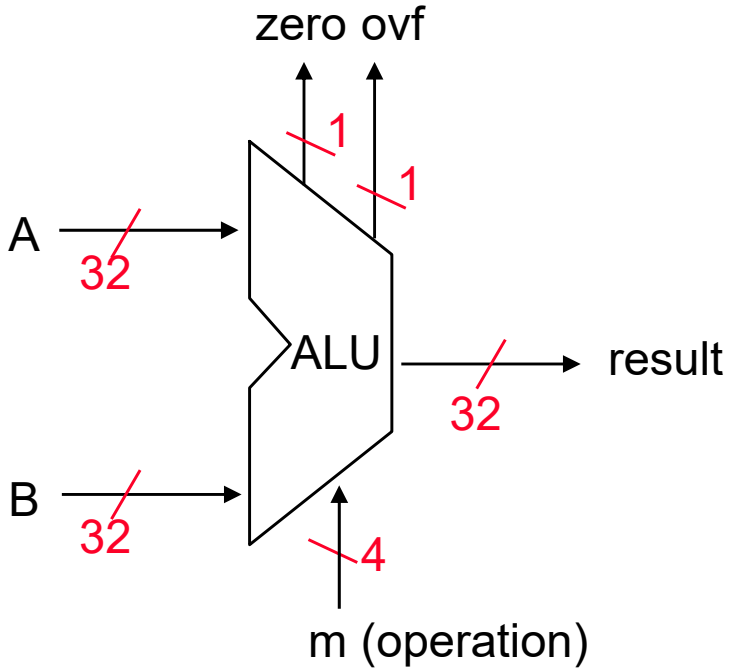
  `ori, xor, xori`

  `mult, multu, div, divu`

  `sqrt`

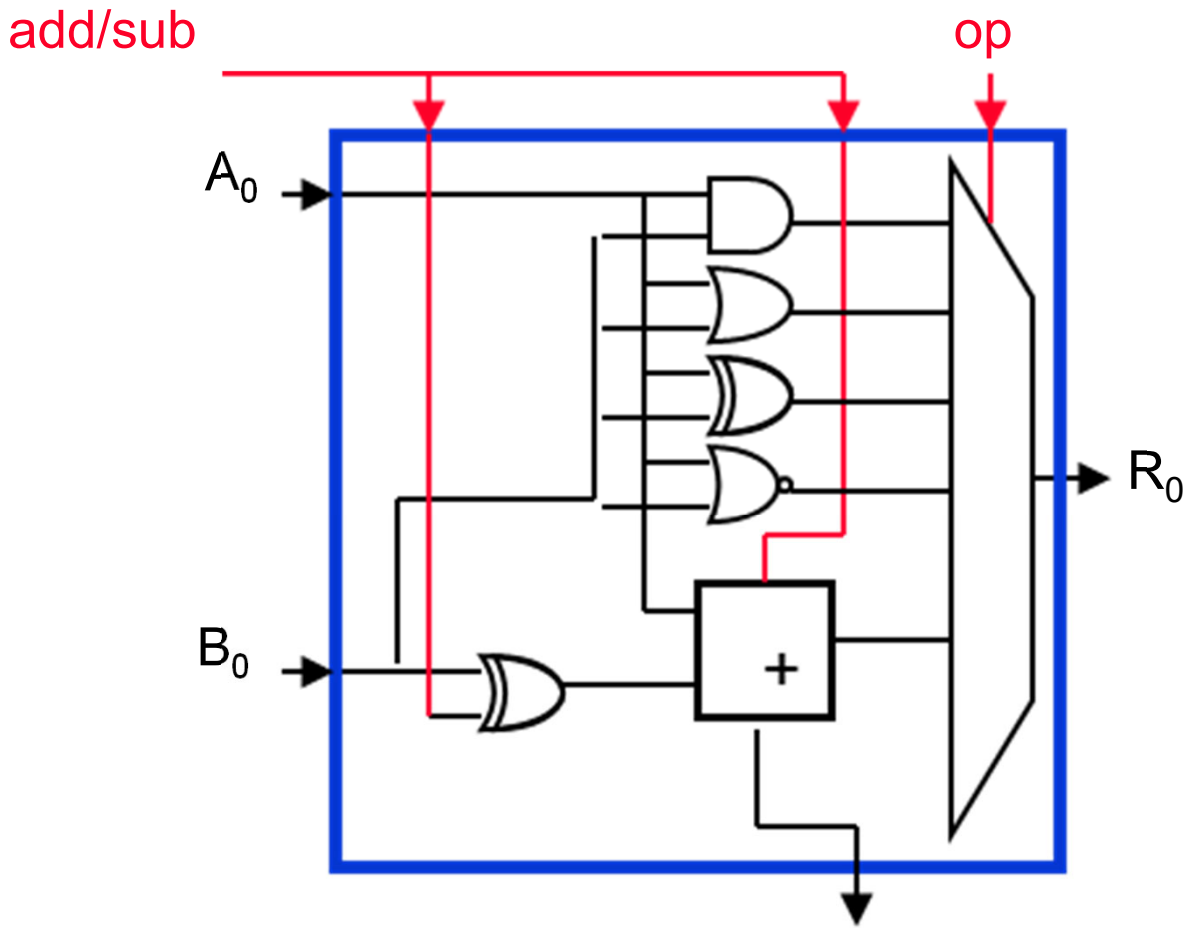
  `beq, bne, slt`

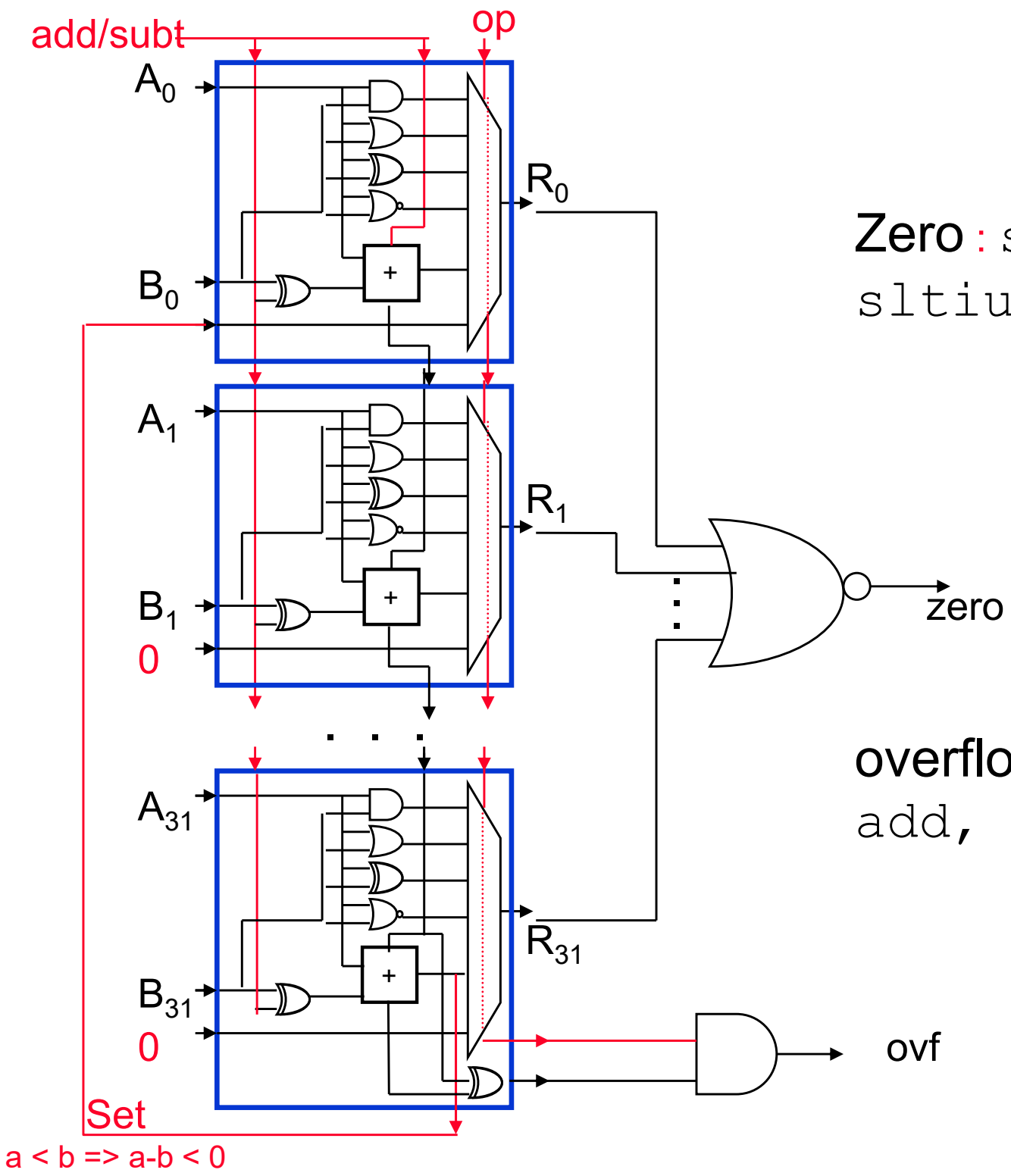
  `slti, sltiu, sltu`



- E inoltre :
  - Estensione di segno – `addi, addiu, slti, sltiu`
  - Estensione con zero– `andi, ori, xori`
  - Rilevamento overflow – `add, addi, sub`

ALU: realizzazione elementare





Zero : `slt, slti, sltiu, sltu, beq, bne`

overflow bit (per operazioni `add, addi, sub`)

Overflow

Overflow quando il carry-in differisce dal carry-out nel bit più significativo (segno)

Sign A	Sign B	Carry In	Carry Out	Sign of result	Correct sign of result	Over-flow?	Carry In XOR Carry Out	Notes
0	0	0	0	0	0	No	0	
0	0	1	0	1	0	Yes	1	Carries differ
0	1	0	0	1	1	No	0	|A| < |B|
0	1	1	1	0	0	No	0	|A| > |B|
1	0	0	0	1	1	No	0	|A| > |B|
1	0	1	1	0	0	No	0	|A| < |B|
1	1	0	1	0	1	Yes	1	Carries differ
1	1	1	1	1	1	No	0	

Carry Lookahead

$c_1 = (a_0 \,.\, b_0) + (a_0 + b_0) \,.\, c_0$

$c_2 = (a_1 \,.\, b_1) + (a_1 + b_1) \,.\, c_1 = (a_1 \,.\, b_1) + (a_1 + b_1) \,.\, ((a_0 \,.\, b_0) + (a_0 + b_0) \,.\, c_0)$

$c_{i+1} = (b_i \,.\, c_i) + (a_i \,.\, c_i) + (a_i \,.\, b_i) = (a_i \,.\, b_i) + (a_i + b_i) \,.\, c_i$

$g_i = a_i \,.\, b_i$ $\qquad\qquad$ $p_i = a_i + b_i$

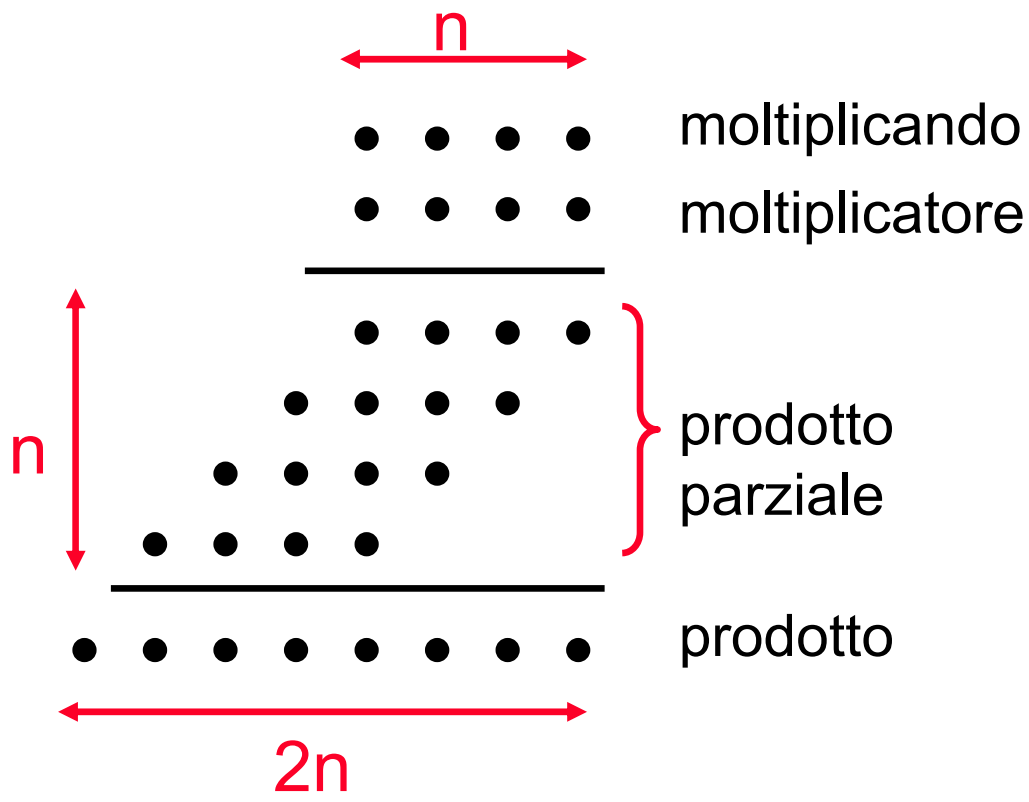
$c1 = g0 + (p0 \,.\, c0)$

$c2 = g1 + (p1 \,.\, g0) + (p1 \,.\, p0 \,.\, c0)$

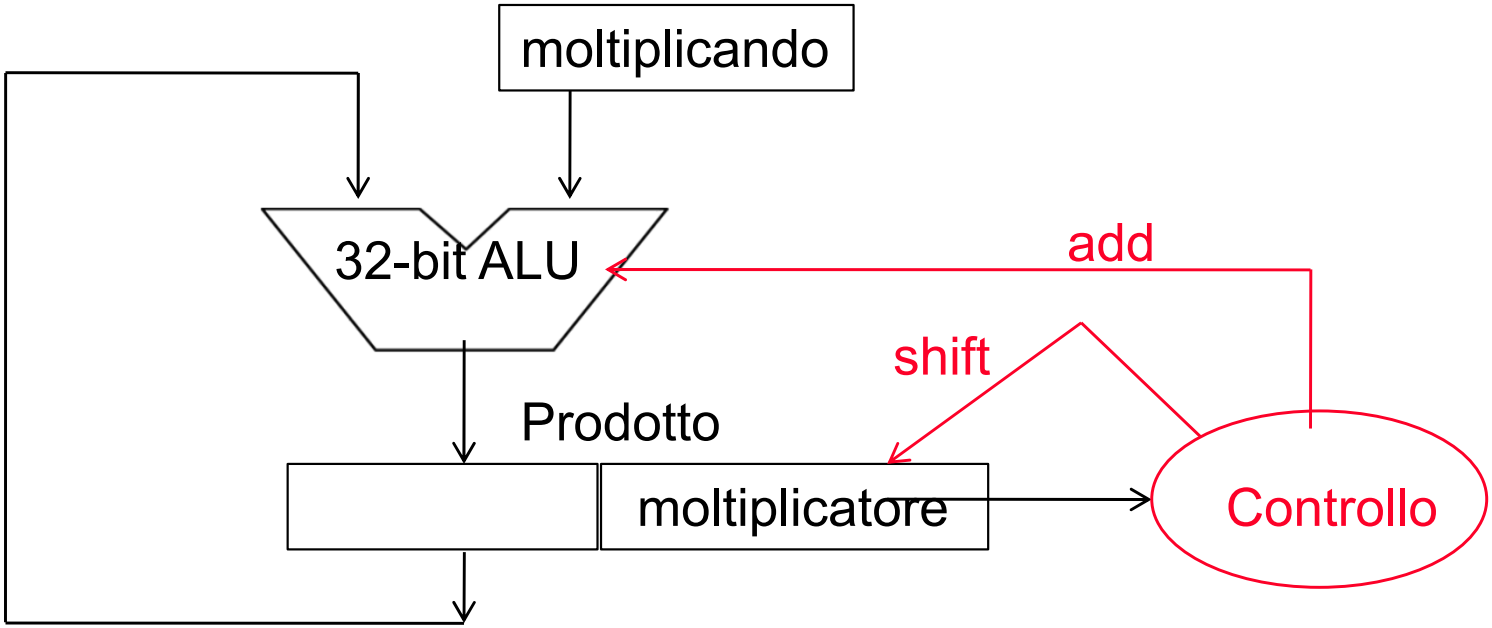
$c3 = g2 + (p2 \,.\, g1) + (p2 \,.\, p1 \,.\, g0) + (p2 \,.\, p1 \,.\, p0 \,.\, c0)$

$c4 = g3 + (p3 \,.\, g2) + (p3 \,.\, p2 \,.\, g1) + (p3 \,.\, p2 \,.\, p1 \,.\, g0) + (p3 \,.\, p2 \,.\, p1 \,.\, p0 \,.\, c0)$

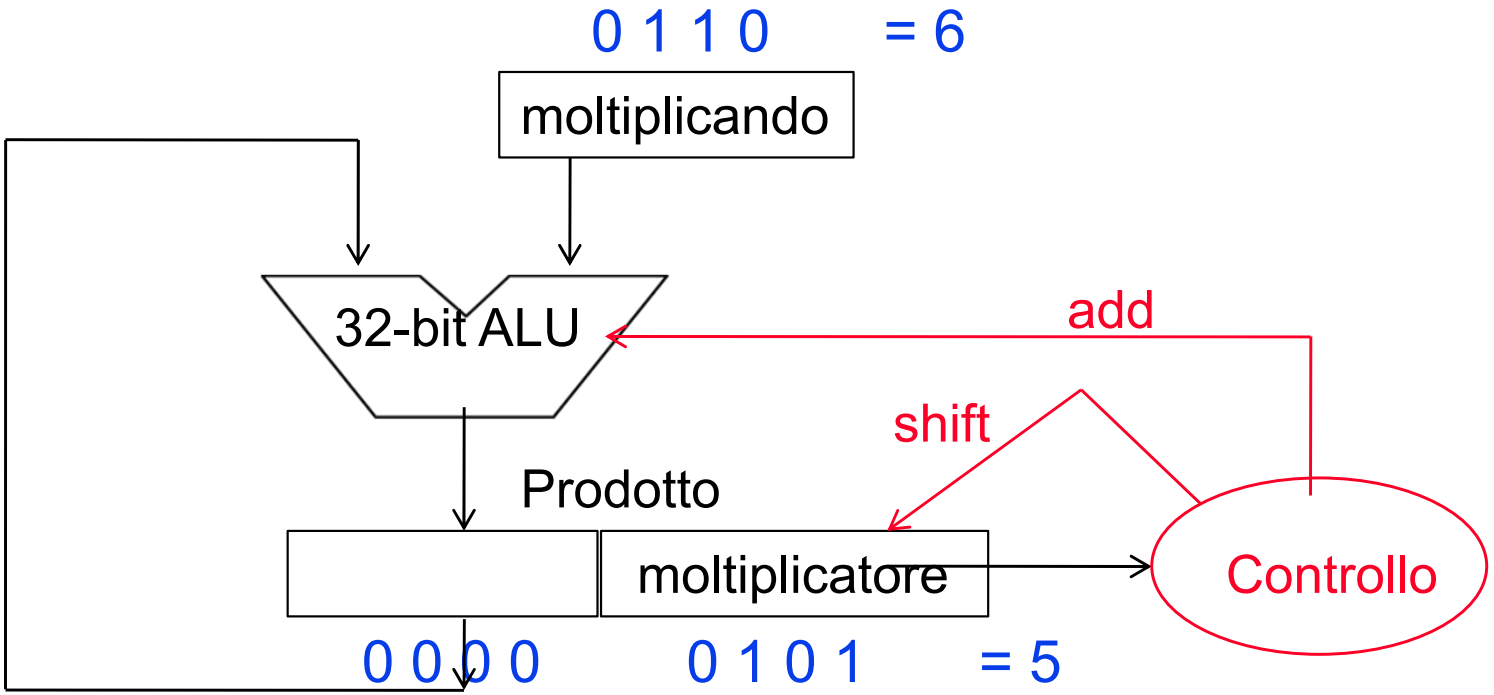
Moltiplicazione



Moltiplicatore “Add and Right Shift”



Moltiplicatore “Add and Right Shift”



0 0 0 0 0 1 0 1 = 5

add 0 1 1 0 0 1 0 1

0 0 1 1 → 0 0 1 0

add 0 0 1 1 0 0 1 0

0 0 0 1 → 1 0 0 1

add 0 1 1 1 1 0 0 1

0 0 1 1 → 1 1 0 0

add 0 0 1 1 1 1 0 0

0 0 0 1 → 1 1 1 0 = 30

Velocizzare la moltiplicazione : algoritmo di Booth

```
      0010two
x     0110two
-------------
+     0000  shift (0 in multiplier)
+    0010   add   (1 in multiplier)
+   0010    add   (1 in multiplier)
+  0000     shift (0 in multiplier)
-------------
  00001100two
```

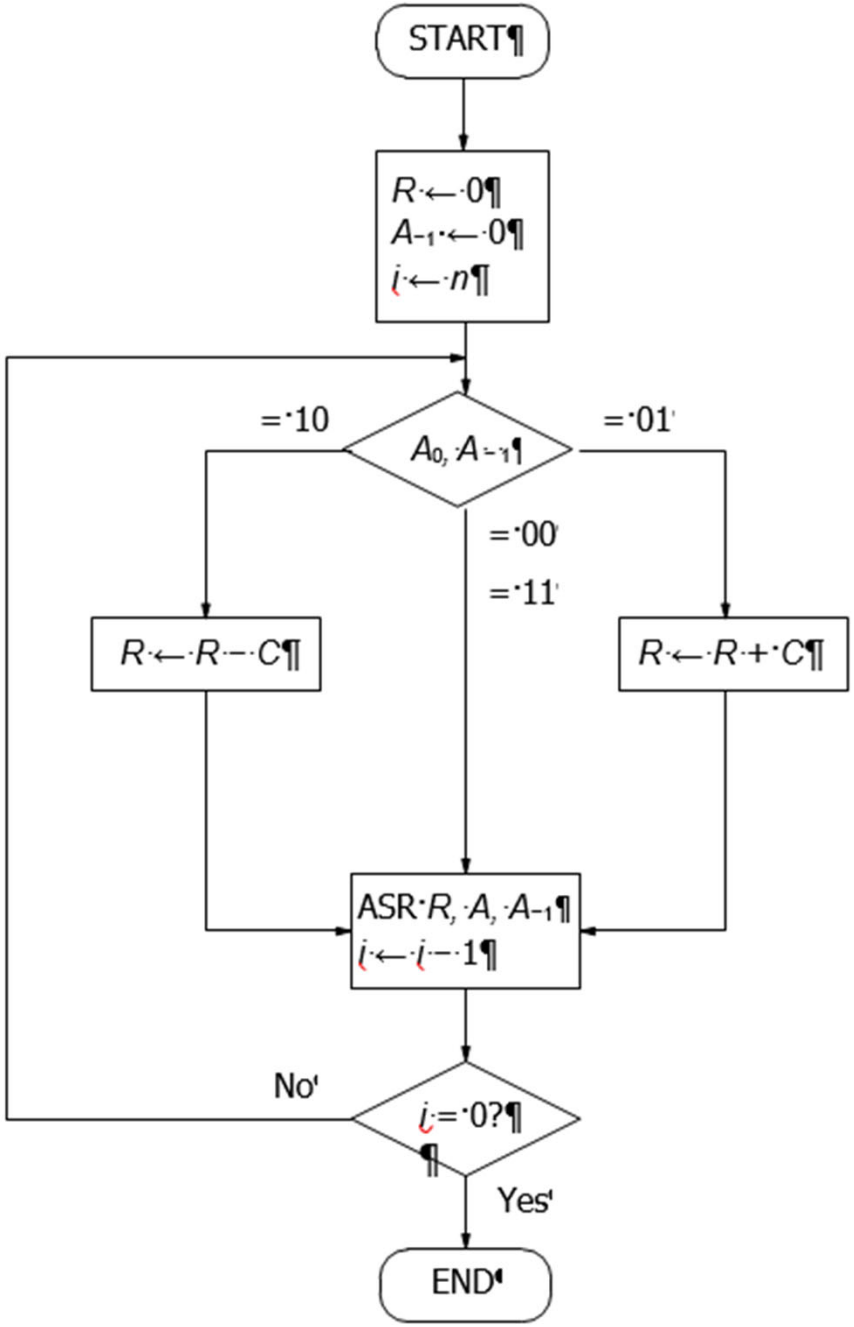
```
      0010two
x     0110two
-------------
+     0000  shift (0 in multiplier)
-    0010   sub (first 1 in multiplier)
+   0000    shift (middle of string of 1s)
+  0010     add (prior step had last 1)
-------------
00001100two
```

0110 = 1000 – 0010

0010*0110 = 0010*1000 – 0010*0010

Itera-tion	Multi-plicand	Original algorithm		Booth's algorithm	
		Step	Product	Step	Product
0	0010	Initial values	0000 0110	Initial values	0000 0110 0
1	0010	1: 0 ⇒ no operation	0000 0110	1a: 00 ⇒ no operation	0000 0110 0
	0010	2: Shift right Product	0000 0011	2: Shift right Product	0000 0011 0
2	0010	1a: 1 ⇒ Prod = Prod + Mcand	0010 0011	1c: 10 ⇒ Prod = Prod – Mcand	1110 0011 0
	0010	2: Shift right Product	0001 0001	2: Shift right Product	1111 0001 1
3	0010	1a: 1 ⇒ Prod = Prod + Mcand	0011 0001	1d: 11 ⇒ no operation	1111 0001 1
	0010	2: Shift right Product	0001 1000	2: Shift right Product	1111 1000 1
4	0010	1: 0 ⇒ no operation	0001 1000	1b: 01 ⇒ Prod = Prod + Mcand	0001 1000 1
	0010	2: Shift right Product	0000 1100	2: Shift right Product	0000 1100 0

Algortimo di Booth



Registri	→	Uso
a0	A	
a1	C	
s1	R	
t0	i	
t1	A_{-1}	(solo bit 0 di t1 è usato)

Algoritmo di Booth: codice assembler

```
            li      a0, -5              # parametro A
            li      a1,  7              # parametro C
            jal     booth

booth:      li      s1, 0               # R <- 0
            li      t1, 0               # A(-1) <- 0
            li      t0,  32             # i  <-  n (32 bits)

loop:       and     t2,  a0, 0x00000001 # t2 <-  A0
            sll     t2,  t2, 1
            or      t2,  t2, t1         # t2 =  A0,A(-1)

            beq     t2,  2, case10      # t2 =  10?
            beq     t2,  1, case01      # t2 =  01?
            j       shift               # t2 =00 or t2 =11  R <-
case10:     sub     s1,  s1, a1         # R <- R -  C
            j       shift
case01:     add     s1,  s1, a1         # R <- R +  C

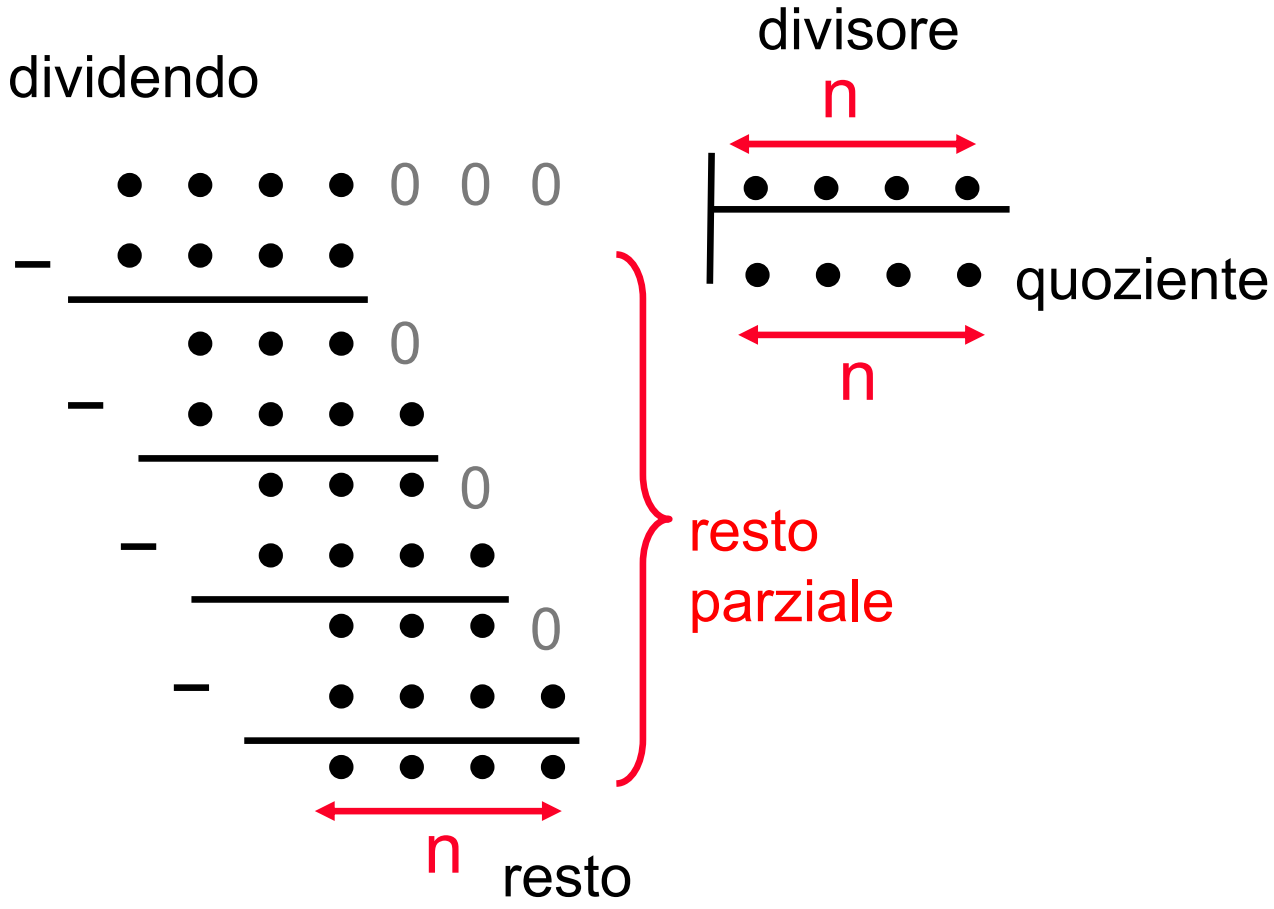
shift:      and     t1, a0, 0x00000001  # A(-1) <- A0
            and     t2,  s1, 0x00000001 # save R0
            sll     t2,  t2, 31
            srl     a0, a0,   1         # shift right A
            or      a0, a0,   t2        # A31 <- R0
            sra     s1, s1,   1         # arithmetic shift right R

            sub     t0,t0, 1            # i  <- i  -  1
            bnez    t0, loop            # i  = 0?
            mv      a0, s1              # risultato in a0
            jr      ra
```

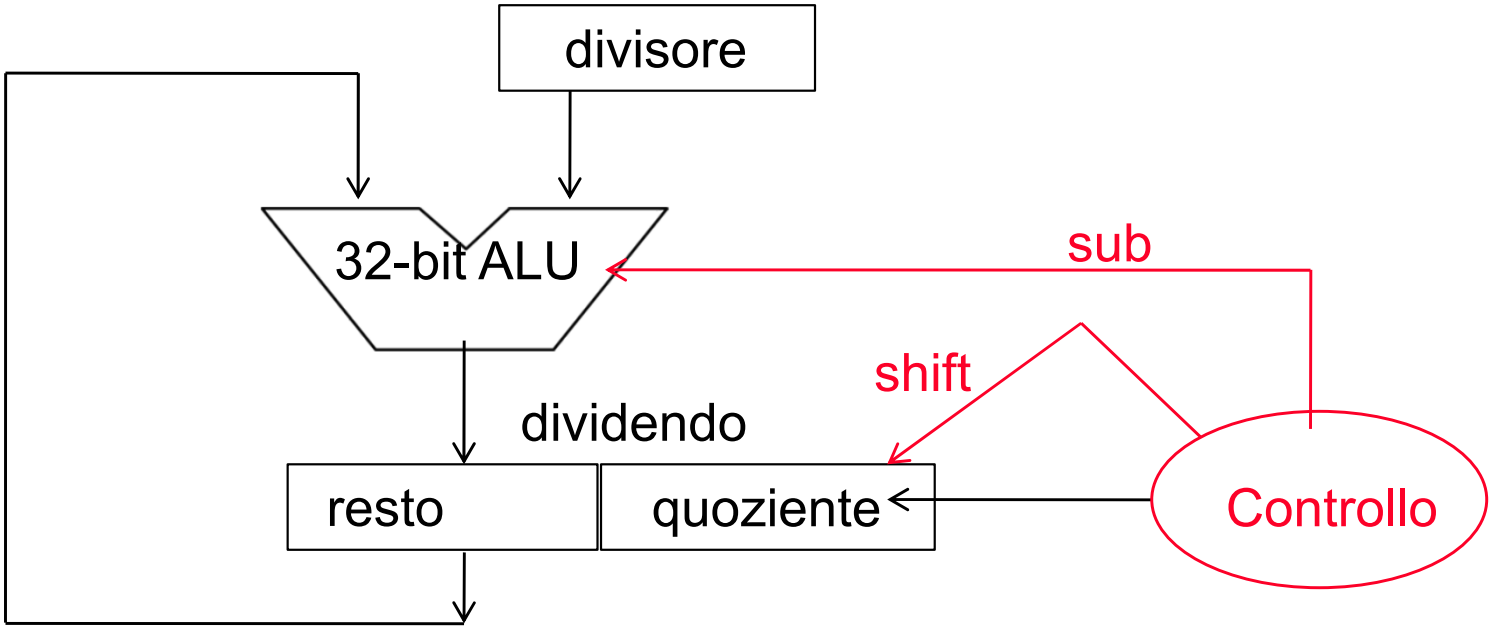
Divisione

- La divisione si compone di scalamenti e sottrazioni

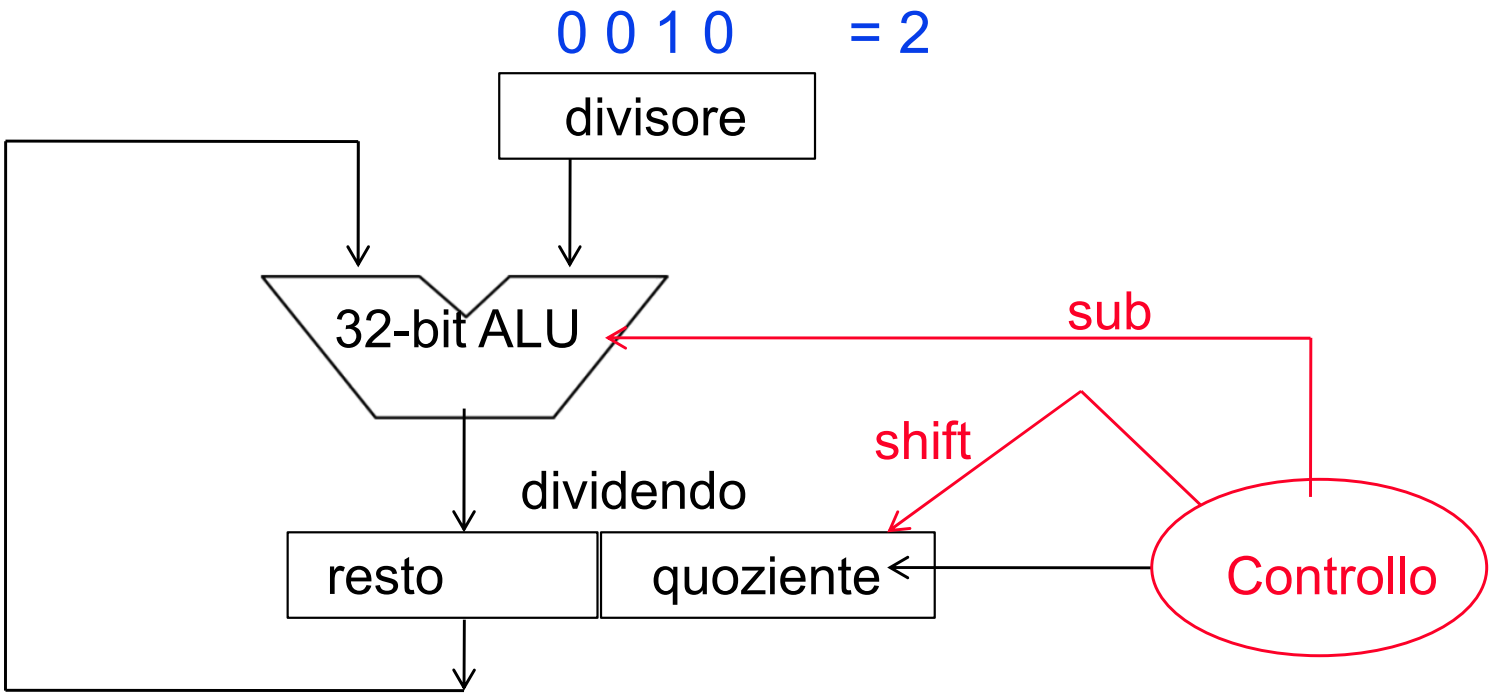
dividendo = quoziente x divisore + resto



Divisione


divisore
32-bit ALU
sub
shift
dividendo
resto
quoziente
Controllo

Divisione




```
      0 0 0 0      0 1 1 0    = 6
      0 0 0 0 ←    1 1 0 0
sub   1 1 1 0      1 1 0 0
      0 0 0 0      1 1 0 0
      0 0 0 1 ←    1 0 0 0
sub   1 1 1 1      1 1 0 0
      0 0 0 1      1 0 0 0
      0 0 1 1 ←    0 0 0 0
sub   0 0 0 1      0 0 0 1
      0 0 1 0 ←    0 0 1 0
sub   0 0 0 0      0 0 1 1
```

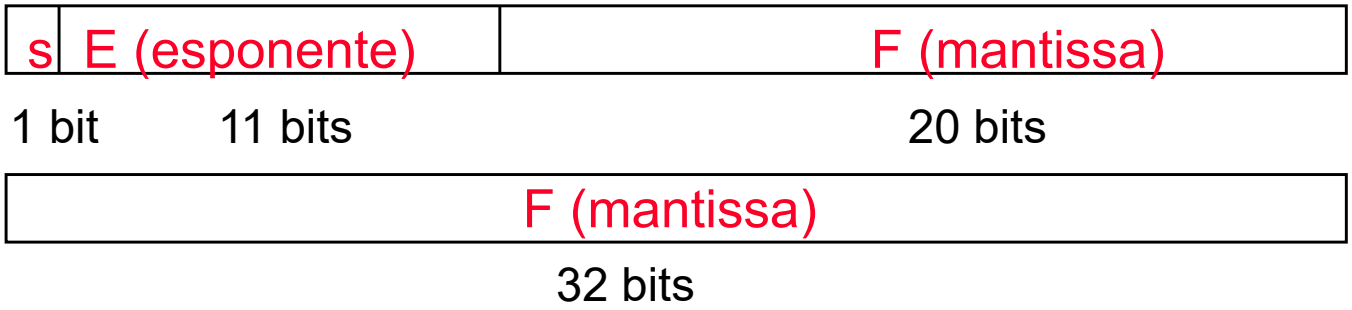
Numeri reali (Floating Point)

- Rappresentazione floating point $(-1)^{sign} \times F \times 2^{E}$
  - singola precisione limitata a 32 bit



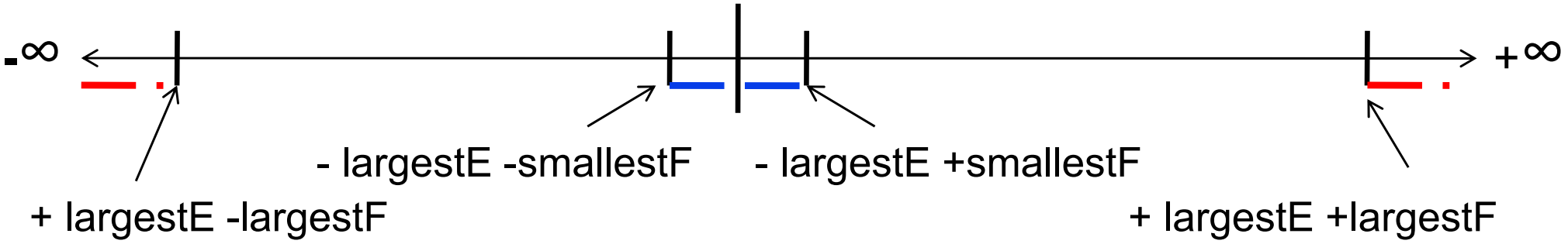
  - La base (2, *non* 10) non è rappresentata ma implicita

  - doppia precisione estesa a 64 bit



Eccezioni

- Overflow quando l'esponente eccede i bit disponibili
- Underflow quando l'esponente di segno negativo eccede I bit disponibili



IEEE 754 FP Standard

$$(-1)^{segno} \times (1+F) \times 2^{E-bias}$$

- Parte frazionaria normalizzata con hidden bit
- Esponente rappresentato con eccesso (biased) :
  - -127 singola precisione
  - -1023 doppia precisione

- Esempi
  - Minimo positivo:

    0 00000001 1.00000000000000000000000 = $1 \times 2^{1-127}$
  - Zero:

    0 00000000 00000000000000000000000
  - Massimo positivo:

    0 11111110 1.11111111111111111111111 = $2-2^{-23} \times 2^{254-127}$
  - $1.0_2 \times 2^{-1}$ = 0 01111110 1.00000000000000000000000
  - $0.75_{10} \times 2^4$ = 0 10000010 1.10000000000000000000000

IEEE 754 FP Standard Encoding

- Codifiche speciali
  - ± infinito (per la divisione con zero)
  - NAN (not a number) per le operazioni non valide (e.g. 0/0)

Singola Precisione		Doppia Precisione		Valore Rappresentato
E (8)	F (23)	E (11)	F (52)	
0000 0000	0	0000 … 0000	0	zero (0)
0000 0000	nonzero	0000 … 0000	nonzero	± numero denormalizzato
0000 0001 0111 1111	numero	0000 … 0001 0111 …1111	numero	± numero
1111 1111	± 0	1111 … 1111	± 0	± infinito
1111 1111	nonzero	1111 … 1111	nonzero	not a number (NaN)

Precisione nelle operazioni

- IEEE 754 : regole di arrotondamento
  - Sempre arrotondato verso l'alto (+∞)
  - Sempre arrotondato verso il basso (-∞)
  - Troncamento
  - Arrotondamento al più vicino (Guard || Round || Sticky bit = 100)
    – inserisce sempre uno 0 nel bit meno significativo di F
- L'arrotondamento richiede la preseza di bit aggiuntivi
  - Guard bit – usato per inserire in F un bit se scalamento a sx per normalizzare un risultato (e.g., dopo una divisione o sottrazione)
  - Round bit – usato per migliorare la precisione
  - Sticky bit – usato per realizzare l'arrotondamento al più vicino; è posto a 1 se un bit=1 scala (dx) attraverso di esso (e.g., per allineare F durante nelle addizioni e sottrazioni)

F = 1 . xxxxxxxxxxxxxxxxxxxxxxx G R S

Addizione

- Addizione (e sottrazione)

$$(\pm F1 \times 2^{E1}) + (\pm F2 \times 2^{E2}) = \pm F3 \times 2^{E3}$$

  - Step 0: Ripristinare hidden bit in F1 e in F2
  - Step 1: Allineare le mantisse scalando a dx F2 di E1 - E2 posizioni (se $E1 \geq E2$) conservare i bit scalati a dx in G R S
  - Step 2: sommare F1 + F2 = F3
  - Step 3: Normalizzare F3 (per ottenere il formato 1.XXXXX …)
    - se F1 ed F2 hanno lo stesso segnon $\rightarrow$ F3 $\in[1,4) \rightarrow$ scalare di 1 bit a dx F3 e incrementare E3 (attenzione a overflow)
    - se F1 ed F2 hanno segni differenti $\rightarrow$ F3 può richiedere diversi scalamenti a sx decrementando ogni volta E3 (attenzione a underflow)
  - Step 4: Arrotondare F3 ed eventualmente normalizzare F3 nuovamente
  - Step 5: Rimuovere il bit (hidden) più significativo di F3 prima di salvare il risultato

Floating Point : esempio addizione

- Add

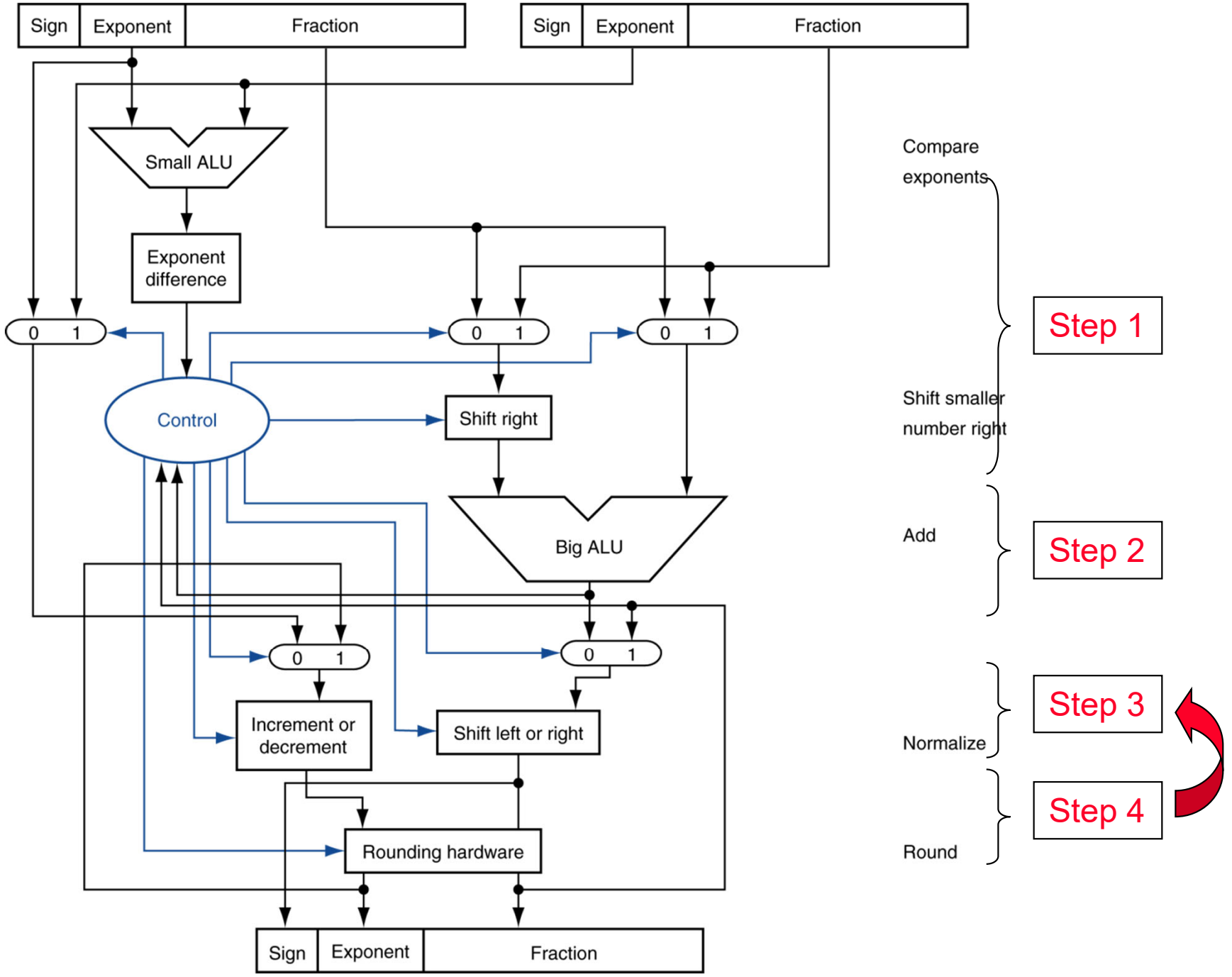
$$(0.5 = 1.0000 \times 2^{-1}) + (-0.4375 = -1.1100 \times 2^{-2})$$

  - Step 0: Reinserire gli hidden bit
  - Step 1: Scalare a dx il numero con esponente minore (1.1100) sino ad uguagliare l'esponente maggiore
  - Step 2: Sommare le mantisse

    $1.0000 + (-0.111) = 1.0000 - 0.111 = 0.001$
  - Step 3: Normalizzare il risultato (verifica over/underflow)

    $0.001 \times 2^{-1} = 0.010 \times 2^{-2} = .. = 1.000 \times 2^{-4}$
  - Step 4: Il risultato non richiede arrotondamento
  - Step 5: Rimuovere hidden bit

FP Adder Hardware



Floating Point : moltiplicazione

- Moltiplicazione

$$(\pm F1 \times 2^{E1}) \times (\pm F2 \times 2^{E2}) = \pm F3 \times 2^{E3}$$

  - Step 0: Reinserire gli hidden bit
  - Step 1: Sommare i due esponenti e sottrarre il bias

    $E1 + E2 - 127 = E3$

    e calcolare il segno del prodotto
  - Step 2: Moltiplicare F1 per F2 = F3
  - Step 3: Normalizzare F3
    - Poichè F1 aednd F2 sono normalizzati allora F3 $\in [1,4) \rightarrow$ scalare di 1 bit a dx F3 ed incrementare E3
    - Verifica overflow/underflow
  - Step 4: Arrotondare F3 ed eventualmente normalizzare F3
  - Step 5: Rimuovere hidden bit

Floating Point : esempio moltiplicazione

- Multiply

$$(0.5 = 1.0000 \times 2^{-1}) \times (-0.4375 = -1.1100 \times 2^{-2})$$

  - Step 0: Ripristino hidden bit
  - Step 1: Somma degli esponenti
    senza bias sarebbe -1 + (-2) = -3
    con il bias risulta (-1+127) + (-2+127) – 127
    = (-1 -2) + (127+127-127)
    = -3 + 127 = 124
  - Step 2: Moltiplicare le mantisse
    1.0000 x 1.110 = 1.110000
  - Step 3: Normalizare il risultato, verificare over/underflow
    1.110000×2^{-3} è già normalizzato
  - Step 4: Il risultato è già arrotondato
  - Step 5: Rimuovere hidden

Frequenza delle istruzioni di uso più comune

	SPECint	SPECfp
addu	5.2%	3.5%
addiu	9.0%	7.2%
or	4.0%	1.2%
sll	4.4%	1.9%
lui	3.3%	0.5%
lw	18.6%	5.8%
sw	7.6%	2.0%
lbu	3.7%	0.1%
beq	8.6%	2.2%
bne	8.4%	1.4%
slt	9.9%	2.3%
slti	3.1%	0.3%
sltu	3.4%	0.8%

	SPECint	SPECfp
add.d	0.0%	10.6%
sub.d	0.0%	4.9%
mul.d	0.0%	15.0%
add.s	0.0%	1.5%
sub.s	0.0%	1.8%
mul.s	0.0%	2.4%
l.d	0.0%	17.5%
s.d	0.0%	4.9%
l.s	0.0%	4.2%
s.s	0.0%	1.1%
lhu	1.3%	0.0%